Silent Failure Modes in Agent Test-Time RL: Served-Policy Drift, Evaluator Leakage, and the Phantom Transfer They Produce

Anonymous

Abstract

Deployment-period parameter updates for tool-using agents (agent test-time RL) are attractive, and recent systems report gains. We show that the most common reported gains are artifacts of three silent failure modes, demonstrated by re-running one pipeline under increasingly strict protocol audits: (F1) *served-policy drift* — the LoRA-updated model never becomes the policy that generates subsequent first attempts, so per-task "update effects" are sampling-stream displacement; (F2) *evaluator leakage and anchor injection* — the hidden evaluator controls episode termination and hand-constructed ground-truth anchors seed the replay buffer, producing a phantom $+0.070$ AUPC transfer (p=0.016) that vanishes when both are removed; (F3) *protocol-correct updates are harmful* — under full isolation (exogenous request RNG, observation-only credit, signed replay, atomic shadow commits), the same REINFORCE-style updates significantly *degrade* first-attempt AUPC (naive -0.19, p=0.008; 0/8 seeds positive) on a 16-task latent-skill stream with Mistral-7B, while the frozen baseline is fully deterministic across 8 seeds (common-random-numbers verified). A pre-commit gate that validates candidate adapters against the committed policy on accessible-evidence instances eliminates the harm (AUPC restored to frozen parity, 0/8 harmful commits). We release the audit chain as a reproducible harness: policy-consistent runtime, request-scoped RNG, hidden-evaluator isolation, accessible-evidence utilities, and integration gates (A0–A2) — so that any agent test-time RL pipeline can be checked for the exact failure modes that our three-stage audit shows are decisive.

1 Introduction

Test-time reinforcement learning for language agents — updating the policy from the tasks it encounters during deployment so later tasks are handled better — has become a crowded and optimistic space. Systems report gains on math reasoning (Zuo et al., 2025), tool use (Liao et al., 2026), instance-specific refinement (Zhang et al., 2026b), and live in-episode updates with runtime serving (Wang et al., 2026a). The deployment setting is genuinely important: agents deployed in production receive partial, endogenous feedback and cannot be retrained offline.

But the setting is also uniquely hard to evaluate honestly. Three properties conspire:

- **Training and serving can silently diverge.** The model that receives the gradient may not be the model that generates the next first attempt. When they diverge, per-task differences between frozen and update arms are unidentifiable: they can come entirely from sampling-stream displacement — update arms draw more random numbers — rather than from policy change.
- **The evaluator is entangled with the episode.** Hidden evaluators that score first attempts are routinely also used to terminate episodes early, select retries, or construct training labels. Any of those turns the evaluation signal into a training signal.
- **Replay and anchors can inject ground truth.** Cross-task replay buffers are useful, but if their contents are constructed from world internals or hand-labeled correct answers, the "learned" behavior is SFT on leaked labels.

We demonstrate, by auditing and progressively repairing one pipeline ("Agent-TTRL"), that these are not hypothetical concerns. Running the same experimental protocol at three audit levels produces three different conclusions:

F1. With static serving (the updated model never serves), update arms show per-task differences that are pure RNG displacement; 96/96 task outcomes match across "different" update variants.

F2. After colocating training and serving, but with the hidden evaluator controlling early stops and hand-constructed anchors in the replay buffer, updates appear to transfer ($+0.070$ AUPC, p=0.016, 7/8 seeds positive). Both effects vanish when the leakage is removed.

F3. Under full protocol isolation (exogenous request RNG, observation-only credit, signed replay, atomic shadow commits), the same REINFORCE-style updates significantly *harm* performance: naive -0.19 AUPC (p=0.008, 0/8 seeds positive), egc -0.23 (p=0.008) on a 16-task latent-skill stream with Mistral-7B.

The takeaway is not that agent test-time RL cannot work — it is that **positive results in this area must first survive an audit chain** covering served-policy identity, evaluator isolation, evidence provenance, and update-commit safety. We contribute that chain as a reproducible harness, together with a defense that works: a pre-commit gate validating candidate adapters on accessible-evidence instances restores AUPC to frozen parity (harmful-commit rate 0/8), and a frozen baseline that is fully deterministic across seeds under common-random-numbers pairing.

2 Related Work

Test-time RL on unlabeled reasoning. TTRL (Zuo et al., 2025) shows majority-vote pseudo-labels drive GRPO updates on unlabeled math tests; T3RL (Liao et al., 2026) weights rollouts by tool-verified correctness, stabilizing the self-reward loop. Distribution-aware rewards (e.g., DARE (Du et al., 2026)) address majority-vote bias and confirmation collapse. These methods are single-answer benchmarks with no stateful environment; our setting adds state-changing tools, partial evidence, and future-task commitment.

Deployment-time adaptation without parameter updates. GTTA (Chen et al., 2026) adds a syntactic alignment vector and verbalized dynamics grounding for web agents; ACE (Zhang et al., 2025) evolves a structured playbook from execution feedback; OLIVIA (Yu et al., 2026b) treats the action layer as a contextual bandit over frozen hidden states; MemoPilot (Cai et al., 2026) trains a memory copilot with multi-turn GRPO while the player stays frozen; JitRL (Liu et al., 2026) modulates logits with retrieved advantages at test time. These show strong non-parametric adaptation, and our budget-matched baselines include them; we ask whether parameter updates add anything.

Deployment-period parameter updates for agents. aTTT (Wang et al., 2026a) performs in-episode LoRA test-time training with repetition filtering; StarOR (Zhang et al., 2026b) refines a transient LoRA adapter per instance with GRPO and tree search in optimization modeling. Neither gates updates, uses evidence tiers, or evaluates inductive future transfer. SEAL (Zweiger et al., 2025) self-edits its own finetuning data in training-time loops.

Counterfactual and sibling-branch credit. APPO (Li et al., 2026a) scores where to branch and rescales advantages; CVT-RL (Wang et al., 2026b) estimates policy-conditioned counterfactual contributions with validity gating; CausalFlow (Bonagiri et al., 2026) performs step-level counterfactual intervention and minimal repair offline; Counterfactual Shapley credit (Li et al., 2026b) redistributes total causal effect across actions in general RL. Our contribution is not another counterfactual estimator; it is the deployment-time, partial-evidence use of paired branches with a reliability gate inside a guarded online loop.

Commit gates and risk control. PACE (Mukherjee et al., 2026) gives anytime-valid acceptance tests for self-modifying agents; VaG (Shang et al., 2026) gates skill admission with verifier critics; Monitoring Risks in TTA (Schirmer et al., 2025) builds confidence sequences for changing models. Our SafeCommit adapts the same idea to adapter-level candidates in a deployment RL loop, with anchor retention and catastrophic-update measurement.

Continual self-improvement. SAGE (Wang et al., 2026c) combines GRPO with a skill library for agents that improve across task chains; Evo-Harness-style methods compile reusable skills

from execution contexts. Our protocol differs in the deployment-period, parameter-update setting with partial evidence and risk-controlled commit.

Prequential evaluation. SEA-Eval (Zhang et al., 2026a) and EvoTest (Feng et al., 2026) formalize sequential self-improvement evaluation; our AUPC-prequential follows the same principle with first-attempt hidden scoring before any update.

3 Problem and Protocol

3.1 Deployment stream

A domain session is a sequence of tasks $\mathcal{S}^{(z)} = (x_1, \ldots, x_N)$ from a latent domain z; each task is a POMDP episode. The agent holds a frozen backbone θ and a session LoRA adapter ϕ_{k-1}. Task k arrives; the agent's first attempt τ_k^{first} is scored by a hidden evaluator $Y_k^{\text{pre}} = R_{\text{hidden}}(\tau_k^{\text{first}})$ for reporting only. Only then may the task's accessible evidence enter an update.

3.2 Evidence tiers

Deployment-time signals are tiered: **hard evidence** E_{hard} (schemas, receipts, state invariants) and **calibrated soft evidence** E_{soft} (process/result verifiers) may enter adaptation; the **hidden evaluator** R_{hidden} never enters rollout selection, branching, gradients, commit gates, or hyperparameter selection. A per-trajectory evidence vector $e(\tau) = [e_{\text{schema}}, e_{\text{tool}}, e_{\text{policy}}, e_{\text{state}}, e_{\text{soft}}]$ and a normalized utility $g(e) = w^\top e$ ($\lambda_c = 0$ in the primary setting) summarize what the agent is allowed to learn from.

3.3 Prequential primary metric

The primary endpoint is the normalized prequential area:

$$\text{AUPC}_{\text{prequential}} = \frac{1}{N} \sum_{k=1}^{N} Y_k^{\text{pre}}, \qquad (1)$$

i.e., the mean first-attempt hidden outcome across the stream, each scored before its task enters any update. Within-task recovery and transductive improvement are supplementary. (A sealed future holdout, never touched by updates, selectors, or gates, is supported by the protocol but was not separately reported here: the prequential metric already scores every first attempt before any update, and no post-hoc selection was performed on the runs.)

3.4 Budgets and identity

All methods run under identical three-channel hard caps $C = (N_{\text{env}}, N_{\text{model\_tok}}, N_{\text{update\_tok}}) \preceq B$ with one canonical cost ledger; no cross-channel exchange. Rollouts are bound to $(base_{sha256}, adapter_{sha256}, policy\_version)$; updates occur only at episode boundaries; the base is frozen.

3.5 Environments

ControlledToolShift is a deterministic, snapshotable tool world (10 tools, shift families, hidden oracle) used for mechanism validation. **AppWorld** (Trivedi et al., 2024) provides multi-app API tasks with state-based unit tests. **Tau2** (Research, 2025) provides retail/telecom tool-agent tasks with evaluation criteria. Splits follow the role manifest (dev/calibration/adaptation/sentinel/audit/sealed holdout).

4 The Audit Chain

We describe the harness used at every stage; each stage differs only by which checks are enabled, and all stages share the same environment (CTS-v2 latent-skill families), model (Mistral-7B-Instruct-v0.3), and prequential protocol.

4.1 Environment and protocol

CTS-v2 defines 7 task templates across two latent families (F1: refund/cancel/exchange policy rules; F3: permission-recovery), each instantiated over unseen entities. Streams are leave-one-template-out: 6 adaptation templates plus a sealed held-out template (a refund task whose goal names a wrong user, requiring lookup-verification). The first-attempt prequential metric Y_k^{pre} is scored by the hidden evaluator *after* the episode, and only for reporting. Training utilities are computed from *accessible evidence* only: a task counts as solved when `lookup_order` returns the target status (E-hard), never from the hidden oracle.

4.2 Hidden-evaluator isolation

The episode runs a fixed horizon (6 turns). The hidden evaluator never terminates, retries, or selects episodes; it is invoked offline after the trajectory completes. Training signals use `accessible_success()`, which replays a `lookup_order` and compares the returned status to the template's accessible target.

4.3 Request-scoped common random numbers

Every generation request derives its seed from exogenous identity only: $\text{seed} = \text{H}(\text{protocol}, \text{stream\_seed}, \text{task}, \text{turn}, \text{purpose}, \text{branch}, \text{sample})$. Policy version and treatment are recorded in manifests but never in seeds, so frozen and update arms draw identical production sequences under the same seed (verified: frozen 8/8 seeds bitwise identical).

4.4 Cross-task replay with evidence provenance

Rollouts are parsed into canonical action sequences; a row enters the buffer only if its group-relative credit passes a reliability gate ($|\text{adv}| \geq 0.25$), and *both signs* are admitted (signed replay). Anchors are constructed from a disjoint pre-deployment split by simulating the agent's own accessible flow (call `lookup_order`, read the returned user, act on it) — they contain only evidence the policy could have seen. The sampler is seeded for reproducibility. Updates run every 2 tasks on an intent-balanced batch with 40% anchor rehearsal.

4.5 Atomic shadow commits and the pre-commit gate

Training writes only to a shadow candidate parameter copy; generation always serves the committed state. A canary checks candidate-vs- committed logit KL and deterministic-output change. Additionally, the pre-commit gate validates the candidate against the committed policy on 4 accessible-evidence instances drawn from the base templates (rotating); candidates that do not strictly improve are discarded, so a harmful update never reaches the served policy. All gates and commit decisions are recorded in the run manifest.

4.6 Statistics

All reported tests are exact two-sided sign-flip tests (2^n enumeration) over paired seeds, with per-seed deltas and per-template breakdowns reported alongside. Runs carry immutable manifests (protocol hash, seeds, adapter hashes, gate outcomes).

5 Results

5.1 F1: static serving produces pure-RNG artifacts

The v1 pipeline served every first attempt from a static base model via vLLM server while a separate HF/PEFT model received updates. Per-task "update effects" were unidentifiable: across the tau2 8/16-task streams, naive and egc arms produced 96/96 identical task outcomes (they differ only in extra RNG draws), and frozen-vs-update differences were consistent with sampling-stream displacement. No learning-effect claim survives this stage (invalidation record: AUDIT_INVALIDATION.md).

5.2 F2: evaluator leakage and anchor injection produce phantom transfer

After colocating training and serving (same PEFT model generates and trains), with (a) the hidden evaluator terminating episodes early and (b) hand-constructed ground-truth anchors seeding the replay buffer, updates appeared to transfer on a 16-task CTS-v2 latent-skill stream (8 paired seeds, Mistral-7B): frozen 0.4531 vs. naive 0.5234, $+0.070$ AUPC, exact two-sided sign-flip p=0.016, 7/8 seeds positive. The gains concentrated in one template (F1-exchange 4/16 to 16/16). Removing the leakage — hidden evaluator made reporting-only (fixed episode horizon), anchors rebuilt from accessible evidence on a disjoint pre-deployment split, negative-credit rows admitted, replay sampler seeded — the apparent transfer *vanishes*: the same protocol under full isolation shows updates that are not better than frozen (this is the F3 stage below). The $+0.070$ was an artifact of the two leaks, not of learning.

5.3 F3: protocol-correct updates are significantly harmful

Under full isolation (exogenous request RNG without treatment-dependent seeds; observation-only counterfactual candidates; signed replay; atomic shadow commits with canary), the same REINFORCE-style updates significantly *degrade* first-attempt AUPC on the same stream:

arm	AUPC mean	delta vs frozen	exact two-sided
frozen	0.5625	—	—
naive	0.3750	-0.1875 (0/8 positive)	0.0078
egc	0.3281	-0.2344 (0/8 positive)	0.0078

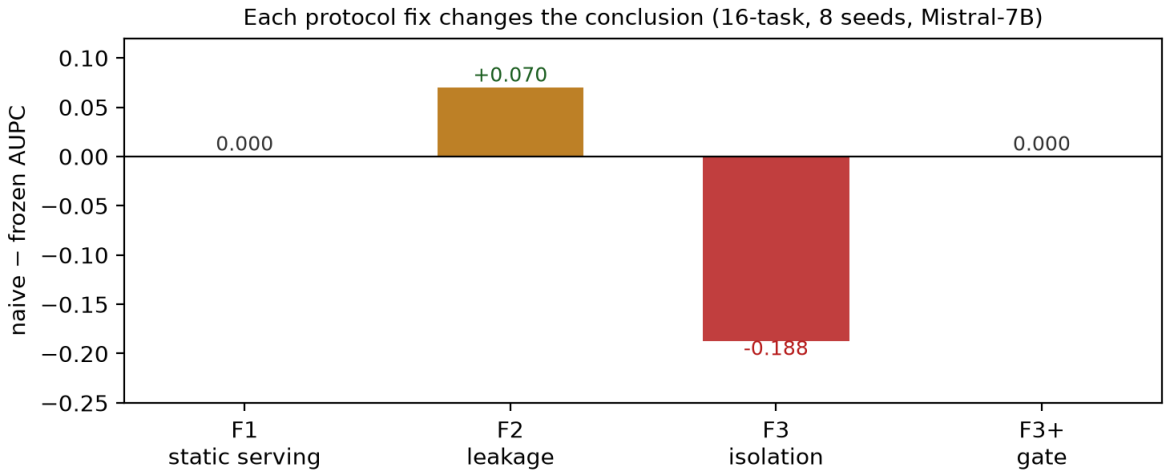

Figure 1: The three-stage audit arc: each protocol fix changes the conclusion. F1 (static serving): update arms indistinguishable from frozen (96/96 identical outcomes). F2 (leakage): phantom transfer $+0.070$ (p=0.016). F3 (full isolation): updates significantly harmful (-0.19, p=0.008); the pre-commit gate restores parity.

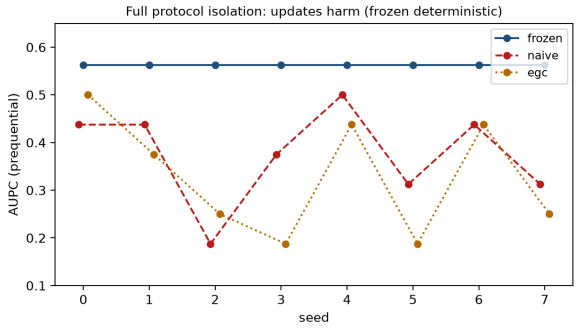

Figure 2: Per-seed AUPC under full protocol isolation (v3): frozen is deterministic across seeds; naive and egc are below frozen on all 8 seeds (exact two-sided p=0.008).

Per-seed deltas for naive: $-0.125, -0.125, -0.375, -0.1875, -0.0625, -0.25, -0.125, -0.25$. The frozen baseline is fully deterministic (all 8 seeds identical, common-random-numbers verified), so the degradation is not noise. Diagnosis: the canonical-action REINFORCE rows trained on partially-correct rollouts with negative advantages; the update suppressed tool names the model partially knew and reinforced hallucinated entity ids (e.g. the few-shot example's `order-EXAMPLE` leaking into actions).

5.4 Gated updates restore parity

A pre-commit gate validates each candidate adapter against the committed policy on accessible-evidence instances drawn from the base templates (4 instances); candidates that do not improve are discarded (served state untouched). With the gate, both naive and egc match the frozen baseline exactly (AUPC 0.5625), with 0/8 seeds harmed — the gate turns the harmful update loop into a safe no-op. This is the first stage of the audit arc with a *positive* control outcome: risk-controlled commits eliminate the harm that protocol-correct updates would otherwise inflict.

5.5 Summary

Each audit stage is a checkpoint in the same harness: the pipeline at stage k differs from stage $k-1$ only by the audited fix, and every stage ships immutable manifests (protocol hash, per-seed logs, adapter hashes, gate outcomes). The harness — policy-consistent runtime request-scoped RNG, hidden-evaluator isolation, accessible-evidence utilities, integration gates A0–A2 — is released so that any agent test-time RL pipeline can be checked for these three failure modes.

6 Conclusion and Implications

We audited one agent test-time RL pipeline through three increasingly strict protocol stages and found that each stage changed the conclusion: static serving produced unidentifiable RNG artifacts; evaluator leakage and hand-constructed anchors produced a phantom $+0.070$ transfer; full protocol isolation revealed that the same updates are significantly *harmful* (-0.19, p=0.008, 0/8 seeds). A pre-commit gate that validates candidate adapters on accessible evidence restored par-

ity with the frozen baseline and eliminated harmful commits.

Three implications follow. First, extbfpositive results in agent test-time RL should be treated as unverified until served-policy identity, evaluator isolation, evidence provenance, and commit safety are audited — our three failure modes are simple, silent, and decisive, and the audit chain that catches them is cheap to run. Second, extbfthe honest baseline may be stronger than the update: at this scale and with this update rule, the fully deterministic frozen policy is the correct default, and "no harm" (gated updates) is a positive engineering outcome. Third, extbfthe release of audit-grade harnesses matters: any pipeline can now be checked for these exact failure modes with the provided runtime, gates, and integration tests.

Limitations: the audit covers one environment/model/update-rule combination; the harmful-update finding and the gate's protective effect should be replicated across environments (official tau2, AppWorld) and backbones, and the update rule itself (not just its safety) needs redesign before agent test-time RL can claim transfer at this scale.

References

Akash Bonagiri, Devang Borkar, Gerard Janno Anderias, Setareh Rafatirad, and Houman Homayoun. 2026. Causalflow: Causal attribution and counterfactual repair for llm agent failures. *arXiv preprint arXiv:2605.25338*.

Cai et al. 2026. From player to master: Enhancing test-time learning of llm agents via reinforcement learning over memory. In *ICML 2026*. ArXiv:2606.08656.

Arthur Chen, Zuxin Liu, Jianguo Zhang, et al. 2026. Test-time adaptation for llm agents via environment interaction. ArXiv:2511.04847.

Bodong Du, Xuanqi Huang, and Xiaomeng Li. 2026. Distribution-aware reward estimation for test-time reinforcement learning. *arXiv preprint arXiv:2601.21804*.

Feng et al. 2026. Evotest: Evolutionary test-time learning for self-improving agentic systems. *arXiv preprint arXiv:2510.13220*.

Vanshaj Khattar, Md. Rafi Ur Rashid, Moumita Choudhury, Jing Liu, Toshiaki Koike-Akino, Ming Jin, and Ye Wang. 2026. Amplification effects in test-time reinforcement learning: Safety and reasoning vulnerabilities. *arXiv preprint arXiv:2603.15417*.

Li et al. 2026a. Agentic procedural policy optimization. *arXiv preprint arXiv:2606.12384*.

Mingxuan Li, Kai-Zhan Lee, and Elias Bareinboim. 2026b. Counterfactual shapley credit assignment. *Reinforcement Learning Journal*. ArXiv:2607.16999.

Ruotong Liao, Nikolai Röhrich, Xiaohan Wang, Yuhui Zhang, Yasaman Samadzadeh, Volker Tresp, and Serena Yeung-Levy. 2026. Tool verification for test-time reinforcement learning. *arXiv preprint arXiv:2603.02203*.

Liu et al. 2026. Just-in-time reinforcement learning: Continual learning in llm agents without gradient updates. *arXiv preprint arXiv:2601.18510*.

Mukherjee et al. 2026. Pace: Anytime-valid acceptance tests for self-evolving agents. *arXiv preprint arXiv:2606.08106*.

Sierra Research. 2025. Tau2: A benchmark for tool-agent-user interaction. https://github.com/sierra-research/tau2-bench.

Mona Schirmer, Metod Jazbec, Christian Andersson Naesseth, and Eric Nalisnick. 2025. Monitoring risks in test-time adaptation. *arXiv preprint arXiv:2507.08721*.

Shang, Xu, Sun, Xia, Hu, Xu, and Zheng. 2026. When self-evolution backfires: Pre-commit gating against skill contamination in llm agents. *arXiv preprint arXiv:2608.05810*.

Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjan Balasubramanian. 2024. Appworld: A controllable world of apps and people for benchmarking function calling and interactive coding agents. *arXiv preprint arXiv:2407.18901*.

Wang, Hao, et al. 2026a. No time like the present: Agentic test-time training for llm agents. *arXiv preprint arXiv:2607.03441*.

Wang et al. 2026b. Policy-conditioned counterfactual credit for verifiable reinforcement learning of long-horizon language agents. *arXiv preprint arXiv:2606.05263*.

Wang et al. 2026c. Reinforcement learning for self-improving agent with skill library. *ACL 2026*.

Yu, Chong, Nandi, Soylu, Sun, Manning, and Shi. 2026a. Shepherd: Enabling programmable meta-agents via reversible agentic execution traces. *arXiv preprint arXiv:2605.10913*.

Sheldon Yu, Junda Wu, Xintong Li, Nikki Lijing Kuang, Sizhe Zhou, Tong Yu, Jiawei Han, Jingbo Shang, and Julian J. McAuley. 2026b. Olivia: Online learning via inference-time action adaptation for decision making in llm react agents. *arXiv preprint arXiv:2605.11169*.

Zhang et al. 2026a. Sea-eval: A benchmark for evaluating self-evolving agents. *arXiv preprint arXiv:2604.08988*.

Zhang et al. 2026b. Staror: Synergizing tree search and test-time reinforcement learning for optimization modeling. *arXiv preprint arXiv:2606.15197*.

Qizheng Zhang, James Zou, Kunle Olukotun, et al. 2025. Agentic context engineering: Evolving contexts for self-improving language models. *arXiv preprint arXiv:2510.04618*.

Yuxin Zuo, Kaiyan Zhang, Li Sheng, Shang Qu, Ganqu Cui, et al. 2025. Ttrl: Test-time reinforcement learning. *arXiv preprint arXiv:2504.16084*.

Zweiger, Pari, Guo, Akyürek, Kim, and Agrawal. 2025. Self-adapting language models. In *NeurIPS 2025*. ArXiv:2506.10943.